

Hybrid Implementation and Evaluation of Classical and Post-Quantum Cryptographic Mechanisms in AI-Based Web Services

Himangi Tripathy¹, Nikita Tosh², Annanya Nayak³, Reetu Kanungo⁴, Rishi Raj Gupta⁵, and Amit Kumar Kar⁶

Department of Computer Science and Engineering,
Siksha 'O' Anusandhan (Deemed to be) University,
Bhubaneswar, Odisha, India

himangiltripathy@gmail.com, nikitatosh@gmail.com, annanyanayak@gmail.com,
reetukanungo@gmail.com, rishi765449@gmail.com, amitkumarkar@soa.ac.in

Abstract. The emergence of large-scale quantum computing presents an existential threat to classical public-key infrastructures, such as RSA and ECC, through Shor's algorithm. This has catalyzed a "Harvest Now, Decrypt Later" threat model, necessitating a transition to NIST-standardized Post-Quantum Cryptography (PQC). However, the increased key sizes and computational demands of PQC algorithms like ML-KEM and ML-DSA create a performance conflict for real-time AI-based web services operating under strict latency budgets. This project proposes PQC-AI, an adaptive cryptographic framework that dynamically selects between classical, hybrid, and post-quantum encryption modes. The system utilizes a Reinforcement Learning agent, formulated as a contextual bandit, to optimize a multi-objective reward function based on real-time CPU load, network conditions, and data sensitivity. By implementing per-sensitivity Double Q-tables, the framework ensures that high-sensitivity data receives robust protection while minimizing overhead for low-risk requests.

Experimental results using liboqs and a fine-tuned DistilBERT sentiment analysis model demonstrate that cryptographic overhead remains below 2.2 ms, even under high concurrency. The RL policy maintains a post-quantum selection rate of over 75%, proving that the dominant bottleneck in intelligent services is AI inference rather than the optimized cryptographic layer. This research provides a practical pathway for achieving cryptographic agility and quantum-resilience in modern digital infrastructures.

Keywords: Post-Quantum Cryptography · Reinforcement Learning · AI Web Services · Cryptographic Agility · Hybrid Encryption

1 Introduction

1.1 Motivation

The security of today's modern digital communication relies heavily on classical public-key cryptosystems such as RSA, Diffie-Hellman, and Elliptic Curve Cryptography(ECC). These algorithms are secure because they rely on computationally intensive mathematical problems that are difficult for current classical computers to solve [10]. However, the emergence of quantum computing can threaten the current encryption system such as Shor's algorithm can solve the mathematical problems underlying RSA and ECC in less time rendering current system vulnerable to quantum attacks [11].

This has catalyzed a "Harvest Now, Decrypt Model" threat model where attacker can store the encrypted data today and decrypt it later using Quantum Computers [4]. To solve this, the National Institute of Standards and Technology(NIST) started a global standardization process for post-quantum cryptographic algorithms. The First finalised standard FIPS 203 selected ML-KEM(derived from CRYSTALS-Kyber) as the main Key Encapsulation mechanism and other finalised algorithm for digital signature includes ML-DSA(Dilithium),Falcon and SPHINCS+. These algorithms use advanced mathematical structures like lattices and hash-based functions to secure against both classical and quantum attacks. While lattice based cryptosystems like ML-KEM performs efficiently using smaller keys, code-based alternatives like McEliece needs larger key sizes whilst providing better security [13].

Even though PQC is more secure, it presents new challenges. In protocols like TLS 1.3, PQC often requires increased computational resources and larger key sizes slowing down TLS handshakes. For example, powerful servers can implement PQC with minimum impact but smaller devices or real time systems become much slower degrading the performance as some algorithms may require 12x more resources than others [2].

In real time environment, such as AI based web services any delay can negatively impact the user experience and service quality. Therefore, a hybrid cryptographic system combining both classical and post-quantum methods are recommended for backward compatibility and "defense-in-depth" during the transition process [3]. Recent research are using reinforcement learning(RL) and artificial intelligence(AI) to dynamically choose between encryption depending upon various performance parameters [2].

1.2 Objectives

Currently, most existing system use static cryptographic configurations without consideration of data sensitivity or real-time operational context [2]. This binary approach creates significant problems i.e., wastes resources on low-risk data as they may use PQC unnecessarily increasing latency, CPU usage and bandwidth consumption or provides insufficient protection for high-sensitive data by defaulting to classical algorithms prioritizing speed over security under heavy load [7]. Even efficient algorithm like Kyber increase TLS handshake size by up to 7x due to its larger key sizes [19].

This is particularly critical for real-time services like healthcare systems, payment applications or AI inference APIs where strict latency must be maintained for fast response time. For example, a single addition of kilobyte(kb) in TLS handshake can increase response time by 1.5% which is intolerable for high velocity transaction systems [6]. Current systems lack a adaptive framework to dynamically select encryption mode based on real-time factors such as system load, network condition, and data sensitivity. Without this adaptable framework organisations are forced to select between performance efficiency and security [24].

This research aims to answer few questions regarding PQC and AI-based adaptive system:

- How efficient are the latest NIST standardized PQC algorithms in comparison to classical algorithms when deployed in real time environment?
- Whether a reinforcement learning based policy help in dynamically optimize cryptographic mode selection in real time?
- To what extent does packet loss and varying network conditions influence the performance of post quantum TLS handshakes?
- Do AI-based adaptive cryptography provides a better trade-off between energy efficiency, latency, and security compared to static cryptographic configurations?

1.3 Original Contributions

This project makes the following contributions:

- Design and Implementation of an adaptive cryptographic framework that dynamically selects between classical, hybrid, and PQC algorithms per request.
- We formulate a sensitivity parameterised reward function that weights security bits against real-time latency and computational costs by introducing the efficiency gap found in static policy.
- We performed a detailed benchmarking of NIST-standardized PQC algorithms in systems where low latency is requirement under varying levels of concurrent user load.
- Our work demonstrates the transition to quantum safe systems by showing that adaptive AI can reduce the slow down caused by hybrid encryption.

1.4 Paper Layout

The remainder of the paper is organized as follows:

Section 2 provides a detailed overview of cryptographic families and identifies research gaps. Section 3 describes the system architecture and RL agent formulation. Section 4 outlines the experimental setup, results, and evaluation. Finally, Section 5 concludes the study and proposes future research directions.

2 Literature Survey

2.1 Overview of Classical Cryptography

Modern digital security infrastructure relies on classical public-key cryptosystems such as RSA, Diffie-Helman, and Elliptic Curve Cryptography(ECC) as they take impractical amount of time to be solved, specifically integer factorization and discrete logarithms [10]. Although current cryptographic systems are secure against classical computational attacks, the emergence of large scale quantum computers present an existential crisis. Shor's algorithm demonstrates that a reliable quantum computer can solve these problems in polynomial time making current public key infrastructure(PKI) vulnerable [19]. This vulnerability has introduced "harvest now, decrypt later" threat model. Here, the attacker can store the encrypted data today and decrypt in once quantum computers become available. Symmetric cryptography is less severely impacted as it relies on key length. For example, the Grover's algorithm only speeds up the brute force attack which can be solved by doubling the key size such as moving from AES-128 to AES-256 [5].

The biggest priority for global security stakeholders is replacing current cryptography with post quantum cryptography. Experts predict that by 2035 quantum hardware will make 2048-bit RSA standards obsolete. Thus, the industry is accelerating the standardization of post quantum algorithms [5].

2.2 Overview of Post-Quantum Cryptography

Post-Quantum cryptography refers to algorithms designed to secure both classical and quantum adversaries. The National Institute of Standards and Technology(NIST) launched a global standardization process in 2016 to secure post quantum algorithms from different mathematical families. After several years to testing NIST selected some official PQC standards such as ML-KEM(derived from CRYSTALS-Kyber) for key encapsulation and ML-DSA(Dilithium) for digital signatures [13]. NIST has also defines 5 security categories ranked in increasing order strength Level 1 security being equivalent to AES-128, Level 2 comparable to difficulty in finding similar hash value in SHA-256/SHA3-256, Level 3 equivalent to AES-192, Level 4 comparable to difficulty in finding similar hash value in SHA-384/SHA3-384, Level 5 security being equivalent to AES-256 [19].

Lattice based cryptography is the most popular PQC approach using problems like Module Learning With Errors(MLWE). ML-KEM is favored due to its relatively moderate key sizes and high execution speed, whereas code based cryptography like McEliece provides stronger security but suffers from larger public key sizes which can exceed 250KB [5]. BIKE and HQC are alternative code based PQC algorithms that offers more balanced trade-off between performance and key sizes [3]. Hash based signatures such as SPHINCS+(SLH-DSA) provides stateless security without depending on factorization or discrete logarithm but has larger signature which makes it slower and requires more computation [9].

Experiments show that Kyber can generate keys much faster than classical RSA. But it still introduces challenges in terms of bandwidth usage. Similarly, PQC implementation increases TLS handshake size but up to 7x which sometimes can lead to packet fragmentation and increased latency in network environment such as mobile networks, IoT devices etc [1]. Also, algorithm like Falcon(FN-DSA) produces compact signatures but are harder to implement securely against side-channel attacks [5]. These characteristics necessitate selection of algorithms based on requirements and constraints of the deployment environment such as device capability, bandwidth, security, etc [13].

2.3 AI-Based Web Services and Security Concerns

Real-time web services using Artificial Intelligence for high velocity tasks operate under strict latency budgets. Integrating PQC into these environment is difficult as the user can degrade because of increase in processing time and memory usage ,for example latency sensitive applications such as video, voice, or digital payments [7].

Experiments reveal that every addition of 1KB of data to TLS handshake increases response time by 1.5% [6]. Naive PQC implementation increases the handshake sizes by 7 times and are slower than pure ECDHE by approximately 22-55%, with BIKE showing the largest increase(55%) which can slow down the the performance

in high concurrency cloud environment. PQC creates challenges under congested or unreliable networks where handshake latency spikes when packet loss occurs. PQC with large message size such as BIKE require up to 20 IP packets to complete a TLS handshake in small MTU settings [1].

For AI services accessed via mobile devices deploying PQC algorithms are very challenging. These challenges are due to hardware limitations, network volatility, and energy sensitivity which increases memory consumption (BIKE often showing 50% higher than Kyber), computational overhead and transmission latency rapidly draining battery resources [2].

2.4 Existing Hybrid Cryptographic Approaches

Hybrid cryptography is recommended to ensure backward compatibility and defense-in-depth during transition to post-quantum encryption. It combines classical algorithms (e.g. ECDHE) with post-quantum schemes (e.g. Kyber), where they operate in parallel to assure key generation speed to be confined by delayed scheme [1]. And remains secure as long as one of the combined algorithm is uncompromised [5].

Trails by leading companies like Cloudflare and Google have demonstrated that hybrid operations (e.g. Kyber+X25519) though doubles the cryptographic workload they only add a marginal latency of approximately 0.3-0.4 ms giving both traditional and quantum security [3]. IETF has also proposed for hybrid key exchange in TLS 1.3 (draft-ietf-tls-hybrid-design) which defines procedure for integrating both classical and PQC algorithms. Some advanced framework uses multifactor authentication and Quantum Random Number Generators (QRNGs) to generate more unpredictable random keys and and strengthen session security [24].

2.5 Gaps in Current Research

Current PQC research treats algorithms (e.g. ML-KEM and ML-DSA) separately from higher level application logic (such as system load, request sensitivity, and latency). While performance for PQC algorithms has already been studied, there is still a lack of research for adapting them dynamically for real time environment taking the system state into consideration. Present PQC deployments often use static policies ignoring dynamic factor such as transaction risk, real time server workload, etc.. For example, using heavy PQC configuration (e.g. ML-KEM-1024 and ML-DSA level 5) for even low risk transaction unnecessary increasing workload, handshake latency, memory usage and slowing down real-time payment processing [7].

Recent literature has explored Reinforcement Learning (RL) for adaptive encryption depending on operational condition but are focused on energy efficiency in Wireless Sensors Networks and IoT devices. For example, Premakumari et al. uses Q-learning to dynamically change encryption level based on operational conditions such as energy consumption, latency, security in constrained network [17]. And there is not established framework that uses Q-learning to dynamically choose between classical, hybrid, and PQC modes for AI-based web services using real time workload and risk-aware conditions [2, 7].

3 Proposed System

PQC-AI is an adaptive web service designed to dynamically apply cryptographic protection during AI inference. This combines sentiment analysis inference with a reinforcement learning based policy engine that selects between classical, hybrid, and post-quantum cryptographic modes according to request sensitivity and current system conditions.

3.1 System Architecture

The PQC-AI service organizes all processing into four modular layers, each of which has its own clear responsibility. Fig. 1 illustrates the complete system architecture. Every inbound HTTP request passes the layers in a defined sequence: API and middleware (Layer 2) → policy engine (Layer 4) → AI inference (Layer 1) → cryptographic layer (Layer 3), with the encrypted response going back to the user in reverse, and it contains encrypted data.

Layer 2 — API and Observability Middleware (FastAPI). API and Middleware is built using the FastAPI Framework. It handles all incoming and outgoing requests of the system. When a request hit the middleware it starts a high-resolution timer, increments a global request counter, and validates the JSON payload using Pydantic schema enforcement. The validated input is described by the schema `{"text": str, "sensitivity": "low" | "medium" | "high"}`. It also checks the current CPU utilisation percentage and tracks a rolling average of end-to-end latency. These state variables are passed to the policy engine prior to inference. On its way out middleware records the total elapsed time, injects an `X-Request-Time-Ms` header into response and sends this latency reading back to the engine. This acts as a scalar reward signal closing the online learning loop.

Layer 4 — Policy Engine (Double Q-Learning). The policy engine observes the current system state and selects a cryptographic action before AI inference begins. It maintains three independent pairs of Q-tables, one pair for each sensitivity level (low, medium, high), and applies an ϵ -greedy exploration strategy. The selected mode `classical`, `hybrid`, or `post_quantum` is forwarded to the cryptographic dispatch table and stored alongside the observed state so that the reward received after request completion can be attributed to the correct state-action pair. The detailed formulation of the state space, action set, and reward function is given in Section 3.3.

Layer 1 — AI Inference Core (DistilBERT). The system uses a pretrained DistilBERT model that handles inference by classifying binary sentiment. The System reads input text from the requested payload and tokenizes it. Then the model processes this data on CPU to simulate a resource-constrained deployment environment. The layer returns a structured result containing the predicted sentiment label (`POSITIVE` or `NEGATIVE`), the associated confidence score, and the measured inference latency in milliseconds.

Layer 3 — Cryptographic Layer. The Cryptographic layer takes the result from the model and serializes it a JSON string. The system encrypts this data using the module selected by policy engine. This layer applies these cryptographic operations using 3 interchangeable modules:

- **Classical mode:** The System uses RSA-2048 with OAEP-SHA256 padding for asymmetric key transport and combines with AES-256-GCM for symmetric authenticated encryption.
- **Post-quantum mode:** This mode relies on ML-KEM-768 (Kyber-768, NIST FIPS 203 [14]) for key encapsulation and ML-DSA-44 (Dilithium3, NIST FIPS 204 [15]) to guarantee digital signature integrity. It pairs these algorithms with AES-256-GCM for symmetric encryption.
- **Hybrid mode:** The system executes of RSA-2048 and ML-KEM-768 key encapsulation concurrently. It then combines the two resulting shared secrets using HKDF-SHA-256 in accordance with NIST SP 800-56C [16] to derive the AES-256-GCM session key. ML-DSA-44 provides the integrity guarantee.

The final response includes the ciphertext, the KEM ciphertext or RSA-encrypted key material, a 96-bit GCM nonce. The system also appends a detached ML-DSA-44 signature in post-quantum and hybrid modes. The complete response bundles the selected cryptographic mode identifier, a policy reasoning string, and the total end-to-end processing time before returning to the client.

3.2 Dataset Description

The binary sentiment classification is executed by components of the AI inference. The model was selected based on its suitability for CPU-based deployment in latency-sensitive settings.

The model used is `distilbert-base-uncased-finetuned-sst-2-english` model. This model is a distilled variant of BERT [25]. It has been fine-tuned on the Stanford Sentiment Treebank (SST-2) [26], which is a widely adopted benchmark for binary sentiment classification. SST-2 contains approximately 67,000 training sentences and 872 validation sentences. Each and every sentence is labelled as either positive or negative. The Hugging Face Hub dataset (`sst2`) does not requires any pre-processing other than tokenization. DistilBERT provides approximately 97% of BERT's performance on the GLUE benchmark with a reduced model

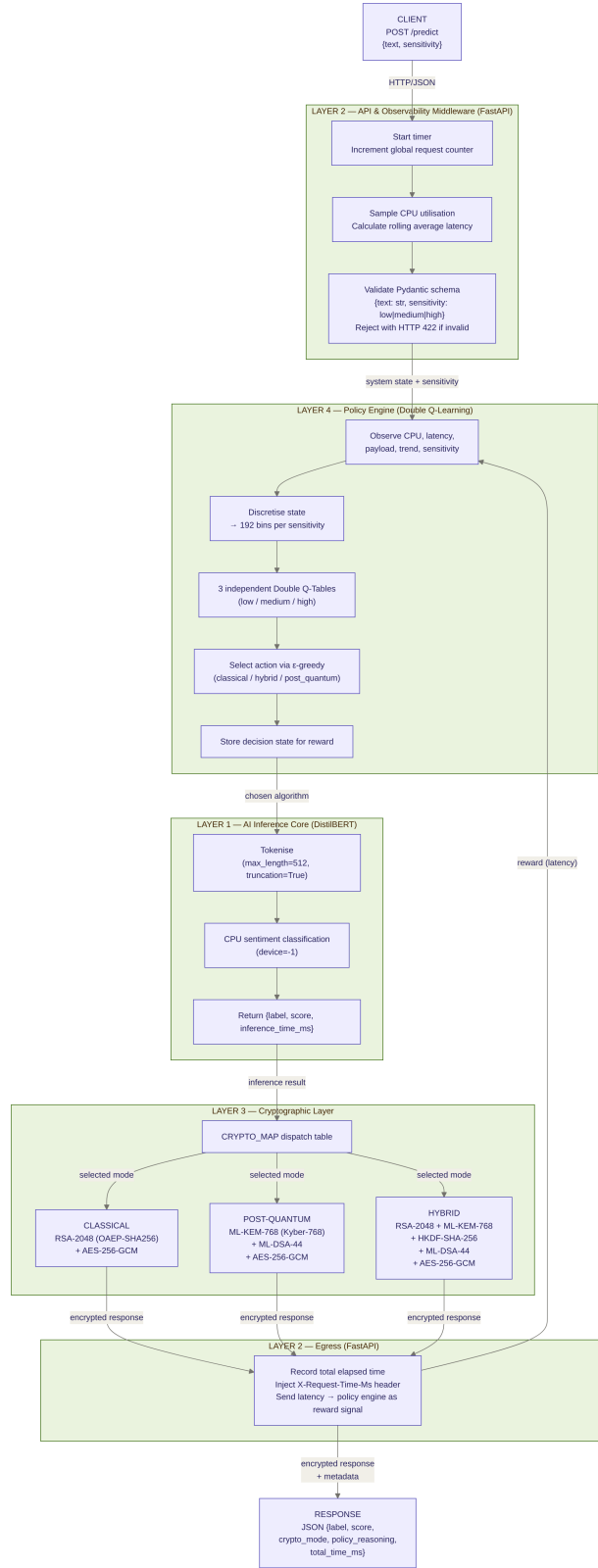


Fig. 1. System architecture of the adaptive PQC-AI web service. Operational flow: (1) FastAPI middleware (Layer 2) validates incoming requests and initializes performance tracking. (2) The policy engine (Layer 4) evaluates system state variables against per-sensitivity Double Q-tables to select a cryptographic mode. (3) The DistilBERT model (Layer 1) executes sentiment inference. (4) The cryptographic layer (Layer 3) encrypts the inference output using the selected protocol (Classical, Post-Quantum, or Hybrid). (5) Middleware records total latency as a feedback reward for online learning and returns the encrypted payload.

size by 40% and inference latency by approximately 60% [18], which makes it well suited to the CPU inference environment this work targets.

DistilBERT tokeniser performs Tokenisation with the following features `max_length=512, truncation=True`, and no additional padding, consistent with the original fine-tuning configuration. The JSON request body provides input text which does not require any normalization without further normalisation. The measured inference latency provides realistic API workloads rather than pre-processed offline inputs. Inference executes with `device=-1`, which forces CPU execution via Hugging Face Transformers' PyTorch backend.

3.3 Methodologies Used

Reinforcement Learning Policy Engine The policy engine is formulated as a contextual bandit. This model is a special case of Q-learning where the discount factor is set to $\gamma = 0$. This setting is justified because a cryptographic action has a negligible effect on future system states. Its primary impact is immediate affecting response latency and CPU utilization. Resulting in optimizing the one-step reward is equivalent to optimizing the long-term objective.

State Representation: The system state $s \in \mathcal{S}$ is defined by five observable features, discretised into a finite set of bins to form a tabular state space. Three independent state-action tables are maintained, one for each sensitivity level. The discretisation scheme is given in Table 1.

Table 1. State space discretisation for the Double Q-learning policy engine. Four CPU bins, four latency bins, four payload bins, and three trend bins yield 192 unique state identifiers per sensitivity table.

Feature	Bins
CPU utilisation (%)	[0–25), [25–50), [50–75), [75–100]
Rolling average latency (ms)	[0–30), [30–80), [80–150), [150–300]
Payload size (bytes)	[0–100), [100–500), [500–2000), [2000+]
CPU trend	falling (< -10), stable ($[-10, 10]$), rising (> 10)
Sensitivity level	low, medium, high (selects table)

The joint state space therefore comprises $4 \times 4 \times 4 \times 3 = 192$ discrete bins per sensitivity level, indexed by the Cartesian product of all feature bins.

Action Space: The agent selects from three discrete actions:

$$\mathcal{A} = \{\text{classical}, \text{hybrid}, \text{post_quantum}\}$$

Double Q-Learning Update: The policy engine maintains two independent value functions, Q_1 and Q_2 , updated alternately with equal probability, to mitigate the overestimation bias inherent in standard Q-learning [23]:

$$\begin{aligned} \text{if rand() } < 0.5 : \quad & Q_1(s, a) \leftarrow Q_1(s, a) + \alpha(r - Q_1(s, a)) \\ \text{else :} \quad & Q_2(s, a) \leftarrow Q_2(s, a) + \alpha(r - Q_2(s, a)) \end{aligned}$$

where $\alpha \in (0, 1)$ is the learning rate and r is the scalar reward received after execution of action a from state s . Because $\gamma = 0$, no bootstrapped target is required. The action selection policy is ϵ -greedy, with ϵ decayed over time to shift from exploration towards exploitation as the agent accumulates experience.

Reward Function: The reward signal R is parameterised by the data sensitivity level, allowing the trade-off between security strength and computational cost to be tuned independently for each tier:

$$R = w_{\text{sec}} \cdot S(a) - w_{\text{lat}} \cdot \min\left(\frac{L}{300}, 1\right) - w_{\text{cpu}} \cdot \min\left(\frac{C}{80}, 1\right) + \delta(a, \text{sensitivity})$$

where L denotes the observed end-to-end latency in milliseconds, C denotes the CPU utilisation percentage at the time of observation, and $\delta(a, \text{sensitivity})$ is a discrete bonus/penalty term applied for high-sensitivity requests. The security score function $S : \mathcal{A} \rightarrow [0, 1]$ is defined as:

$$S(\text{classical}) = 0.4, \quad S(\text{hybrid}) = 0.7, \quad S(\text{post_quantum}) = 1.0$$

The weight vector $(w_{\text{sec}}, w_{\text{lat}}, w_{\text{cpu}})$ is sensitivity-dependent: high-sensitivity configurations assign greater weight to w_{sec} , penalise the selection of weaker cryptographic modes, and reward the selection of `post_quantum`. The total system latency is decomposed as:

$$T_{\text{total}} = T_{\text{inf}} + T_{\text{crypto}} + T_{\text{policy}}$$

where T_{inf} is DistilBERT inference time, T_{crypto} is cryptographic processing time, and T_{policy} is the policy engine decision overhead.

Cryptographic Mode Selection The policy engine’s decisions are data-driven and emerge from reward optimisation rather than hard-coded thresholds. Table 2 provides an illustrative summary of the expected dominant mode under representative system conditions.

Table 2. Illustrative mapping from system state and sensitivity to expected dominant cryptographic mode. Actual decisions are learned online by the policy engine.

System State	Sensitivity	Expected Mode
Low CPU, low latency	Low	Post-quantum (security favoured)
High CPU, high latency	Low	Classical (latency favoured)
Any load	High	Post-quantum (bonus + penalty)
Moderate CPU, moderate latency	Medium	Hybrid (balanced)

API Endpoint Structure The PQC-AI service exposes a RESTful HTTP interface via FastAPI. The principal endpoints are described in Table 3.

The `/predict/encrypted` endpoint implements a full client-side encapsulation workflow: the client first retrieves the server’s public KEM key from `/keys/{mode}`, performs local key encapsulation to obtain a shared secret, encrypts the plaintext with AES-256-GCM under that secret, and submits the resulting ciphertext and KEM ciphertext to the endpoint. This design ensures that plaintext is never transmitted in the clear.

Post-Quantum Cryptographic Integration via liboqs The post-quantum primitives are provided by the Open Quantum Safe library `liboqs` (v0.15.0) [20], accessed through the official Python binding `liboqs-python` (v0.14.1). A wrapper class, `PostQuantumCrypto`, encapsulates all key generation, encapsulation, signing, and decryption operations.

Key Initialisation: Key pairs for the KEM and signature schemes are generated at service startup:

Table 3. REST API endpoint specification for the PQC-AI service.

Endpoint	Method	Description	Request Body	Response Schema
/predict	POST	Plaintext inference with policy-selected crypto metadata	{"text": str, "sensitivity": str}	{label, score, crypto_mode, times_ms, reasoning}
/predict/encrypted	POST	End-to-end encrypted inference	{"sensitivity": str, "kem_ciphertext_hex": str, "nonce_hex": str, "ciphertext_hex": str}	{encrypted_result, kem_ciphertext}
/keys/{mode}	GET	Retrieve server public keys for client-side encapsulation	—	{kem_public_key_hex, sig_public_key_hex, rsa_pem}
/health	GET	Service liveness check	—	{"status": "healthy", "model_loaded": bool}
/metrics	GET	Aggregate request-level performance statistics	—	{"total_requests": int, "avg_latency_ms": float}
/policy/stats	GET	Policy engine internal state snapshot	—	{distribution, epsilon, q_table_updates}
/policy/history	GET	Time-series of rewards, epsilon decay, and per-request decisions	—	{"rewards": {...}, "epsilons": {...}, "decisions": [...]}

```

with oqs.KeyEncapsulation("Kyber768") as kem:
    self.kem_public_key = kem.generate_keypair()
    self.kem_secret_key = kem.export_secret_key()

with oqs.Signature("ML-DSA-44") as sig:
    self.sig_public_key = sig.generate_keypair()
    self.sig_secret_key = sig.export_secret_key()

```

Encryption Procedure: The `encrypt` method executes the following steps in sequence: (i) validation of the encapsulation key against the checks specified in FIPS 203 §7.2 [14]; (ii) key encapsulation via `kem.encap_secret(public_key)` which returns a KEM ciphertext and a shared secret; (iii) AES-256-GCM key derivation from the shared secret; (iv) authenticated encryption of the serialised inference result; and (v) detached digital signature of the ciphertext using ML-DSA-44. The method returns a structured dictionary containing the ciphertext, KEM ciphertext, 96-bit nonce, signature bytes, and per-operation timing measurements.

Decryption Procedure: The `decrypt` method first verifies the ML-DSA-44 signature. Upon successful verification, it calls `kem.decap_secret(kem_ciphertext)` to recover the shared secret, re-derives the AES key, and decrypts the payload using AES-256-GCM authenticated decryption.

Hybrid Key Derivation: In hybrid mode, both the Kyber-768 and RSA-2048 encapsulation operations are performed concurrently. The two resulting shared secrets, K_{PQC} and K_{RSA} , are concatenated and passed through HKDF-SHA-256 in accordance with NIST SP 800-56C [16] to derive the final AES-256-GCM session key:

$$K_{\text{session}} = \text{HKDF-SHA256}(K_{\text{PQC}} \parallel K_{\text{RSA}}, \text{salt}, \text{info})$$

This construction ensures that the session key is computationally secure provided that either the classical or the post-quantum component remains unbroken, thereby providing a cryptographic security guarantee that is no weaker than either constituent scheme.

4 Experimentation and Model Evaluation

This section presents the experimental setup used to evaluate the proposed adaptive PQC-AI system. The experimental setup, testing methodology, system performance evaluation, contributions, and results obtained from various experimental scenarios are discussed. It also evaluates the effectiveness of the reinforcement learning-based policy engine and analyzes the overall system performance.

4.1 Experimental Setup

The experimental evaluation of the adaptive PQC-AI system was conducted in two phases: (i) a cryptographic microbenchmark that measures the isolated per-operation latency for each of the three cryptographic modules and (ii) a load-testing phase that evaluates the end-to-end system performance under simulated concurrent user traffic.

All experiments were conducted on a single machine equipped with an Intel Core i5 CPU and 16 GB RAM, using Ubuntu 24.04 LTS as the operating system. The PQC-AI architecture was deployed using Docker Compose with two separate services: a FastAPI inference server and an Nginx reverse proxy server. The former had a configuration for a self-signed 2048-bit RSA TLS certificate, which enabled standalone testing of the HTTP and HTTPS protocols. The server was run with a single worker of Uvicorn to avoid including the latency introduced by scheduling in the performance measures. The inference model, `distilbert-base-uncased-finetuned-sst-2-english` [18, 25, 26], was loaded on initialization and bound to the CPU only (`device = -1`). Post-quantum primitives were implemented using `liboqs` (v0.15.0) [20].

Cryptographic Microbenchmark. All three cryptographic modules (classical (RSA+AES), post-quantum (ML-KEM and ML-DSA), and hybrid) were evaluated through 100 test runs for each module using a 63-byte payload size. This was chosen specifically to mimic a typical serialized DistilBERT inference result. The mean, standard deviation, and extremes were captured for both the encryption and decryption. This isolated benchmark ensured that the AI inference time did not skew the results.

Load Testing Framework. The system-level performance was evaluated using an open-source HTTP load-testing framework called Locust. Users were divided into two classes: SentimentUser, which simulates realistic API traffic with a task weight distribution of medium-sensitivity requests (weight 5), high-sensitivity requests (weight 2), low-sensitivity requests (weight 1), and monitoring endpoints (weight 1 each), and HeavyLoadUser, which submits low-sensitivity stress-test requests at a shorter inter-request interval (0.1–0.5 s) to saturate the server. The system was tested with four different numbers of users -15, 35, 55, and 80. Tests were performed in headless mode for a fixed run duration for both plain HTTP and TLS-enabled (HTTPS) endpoints. The results were exported as CSV statistical files for analysis.

Policy Engine Configuration. The reinforcement learning policy engine was implemented as described in Section 3.3.1. The engine is a contextual bandit where $\gamma = 0$ with per-sensitivity Double Q-tables [23]. It operates in a state space composed of 192 states. It is divided in terms of four observable variables: CPU utilization, rolling average latency, packet size, and CPU trend. An ϵ -greedy schedule with an initial exploration rate of $\epsilon = 0.30$ was used with a multiplicative decay of 0.999 per decision until reaching a floor of 0.05 [21]. Experience replay buffers were also implemented for each sensitivity setting [12] of buffer size 5,000, mini-batch size 32, and trained every 50 decisions. This was done to ensure the separation of subsequent Q-table updates. Another aspect was re-exploration, where ϵ would be increased to 0.20 whenever the reward of the previous 100 decisions fell below one standard deviation below the mean.

Table 4. Cryptographic module microbenchmark results (100 iterations, 63-byte payload).

Algorithm	Enc. Mean (ms)	Enc. Std. Dev. (ms)	Dec. Mean (ms)	Dec. Std. Dev. (ms)
Classical (RSA-2048 + AES-256-GCM)	0.336	0.315	1.836	0.579
Post-Quantum (ML-KEM-768 + ML-DSA-44[?] + AES-256-GCM)	1.093	0.276	0.298	0.078
Hybrid (RSA-2048 + ML-KEM-768 + HKDF-SHA-256[?] + AES-256-GCM)	1.537	0.292	2.142	0.409

4.2 Results and Outputs

The microbenchmark results for all three cryptography modules are presented in Table 4. Although classical encryption provides the least latency in terms of encryption (0.336 ms), the same type requires the largest decryption overhead (1.836 ms), which can be associated with RSA-2048 private key operations. On the other hand, the post-quantum cryptography module has a medium overhead for encryption (1.093 ms) owing to the ML-KEM-768 key encapsulation procedure; however, it is the most efficient module in terms of decryption latency (0.298 ms) compared to RSA private key exponentiation. The hybrid solution requires the largest overhead for both encryption and decryption types (1.537 ms and 2.142 ms, respectively). However, overall, the cryptographic latency for each operation does not exceed 2.2 ms, indicating that the overhead caused by cryptography is relatively small compared to the inference time of DistilBERT.

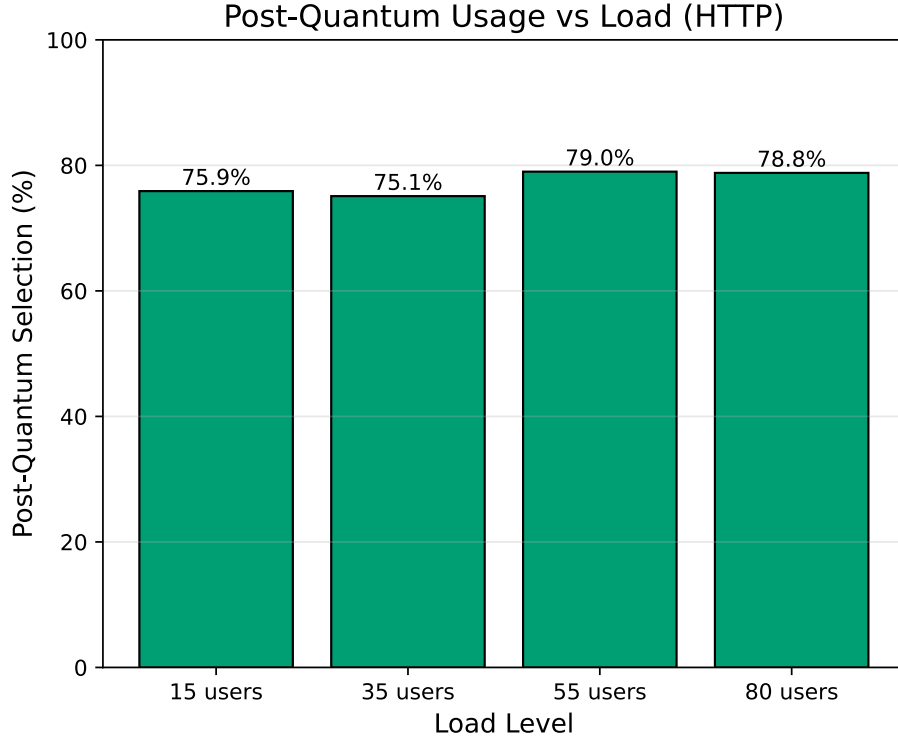
Load Test Results. Table 5 summarizes the aggregate end-to-end performance results obtained from Locust in each of the four load scenarios with the HTTP transport protocol setting.

Under the load of 15 users, the system behavior was relatively stable, with no errors occurring, a median response time of 1,300 ms, and a throughput of 7.05 rps. The majority of latency is caused by the inference of

Table 5. Aggregated load test results across four concurrency levels (HTTP, adaptive policy).

Load	Avg. Latency (ms)	Median Latency (ms)	Requests/s	Failures	Failure Rate (%)
15 users	1,420	1,300	7.05	0	0.0%
35 users	3,647	3,500	7.46	24	1.8%
55 users	8,950	8,200	5.05	249	27.1%
80 users	9,141	6,700	7.08	261	20.2%

DistilBERT, while cryptography takes up less than 2 ms. The decline in system performance starts at 35 users, resulting in a latency of 3,500 ms and timeouts of 24 requests, which amounts to 1.8%. CPU queueing started to affect the results. At 55 users, the system demonstrated significant instability, with the median response time rising to 8,200 ms and 27.1% of requests timing out. The identified behavior corresponds to CPU saturation due to the competition of calculations done on DistilBERT with those made by stress requests. The slight decrease in failure percentage under the load of 80 users (20.2%) does not imply any improvement. This was caused by retrying failed requests by Locust under full server CPU power usage.

**Fig. 2.** Fraction of Post-Quantum Algorithm Choices Under Four Levels of Load Using HTTP Transport. These numbers represent the fraction of post-quantum choices out of the most recent 1,000 choices made at each corresponding load level.

The smoothed learning curve for the medium-sensitive Q-table at a 55 user load is depicted in Figure 3 (averaged over 50 requests). The reward begins from roughly -0.16 , falls down to a minimum of approximately -0.28 at request number 100 owing to the exploratory nature of the ϵ -greedy algorithm [21], and finally recovers, reaching almost constant levels of -0.14 by request 300. This trend fits into what can be seen as the standard

procedure for online reinforcement learning, when a short exploratory phase followed by Q-value updates eventually results in finding optimal actions.

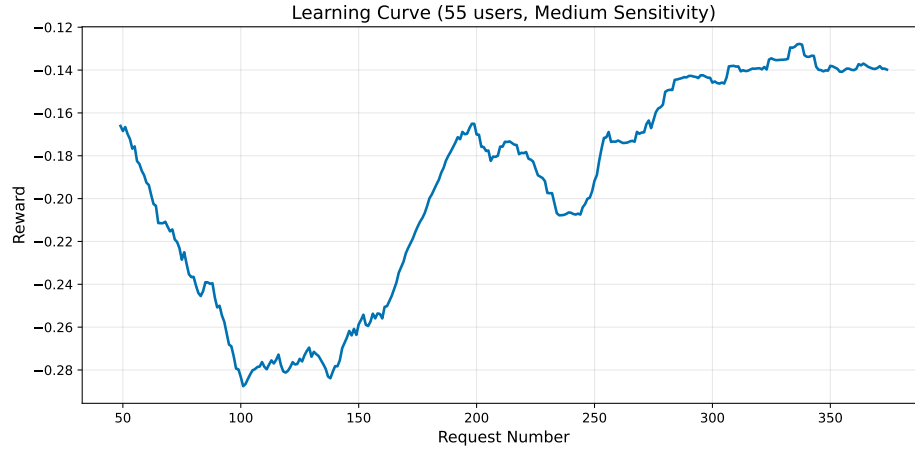


Fig. 3. Medium Sensitivity Q-Table Learning Curve – 55 concurrent users – 50 Request Rolling Average. The reward is graphed against the request number; a larger value indicates better policy behavior.

The behavior of the algorithm selection frequencies over time is shown in Figure 4. In the case presented here, there are 55 active users, and the time windows used consist of 100 requests each. As can be observed from the figure, the selection frequencies of the post-quantum approach remain dominant throughout the course of the experiment, except for rare instances of selection frequency for hybrid and classical algorithms when exploration is increased or when re-exploration is performed owing to a reduction in rewards.

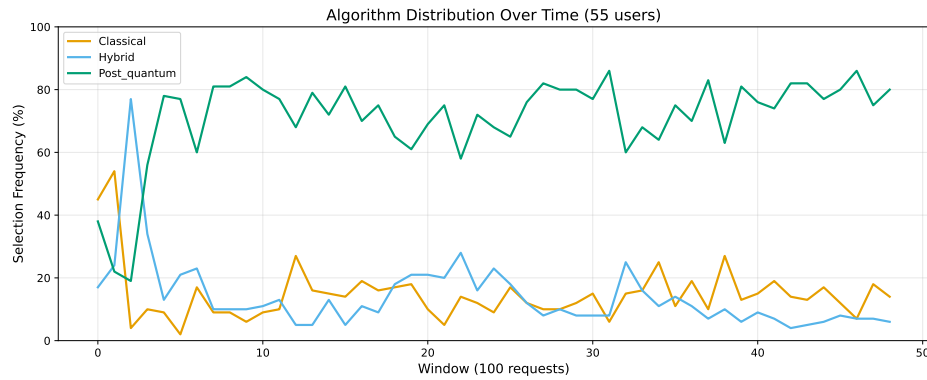


Fig. 4. Number of times the algorithm was selected over time by 55 users simultaneously. Each dot represents the proportion of the cryptographic algorithm selected during each set of 100 requests.

Figure 5 shows a scatter plot of the percentage CPU utilization at the decision points against the number of requests, classified according to the algorithm selected based on the first 200 selections per algorithm with 55 concurrent users. In all three cases, algorithms were selected over a similar range of CPU utilization levels (approximately 40% to 67%) without any distinct cutoff, where classical algorithms were only selected under

lower CPU utilization levels or post-quantum algorithms were only selected under higher CPU utilization levels. This suggests that the policy engine operates based on a multidimensional reward function involving latency, CPU utilization, and security weightings rather than a threshold CPU utilization level.

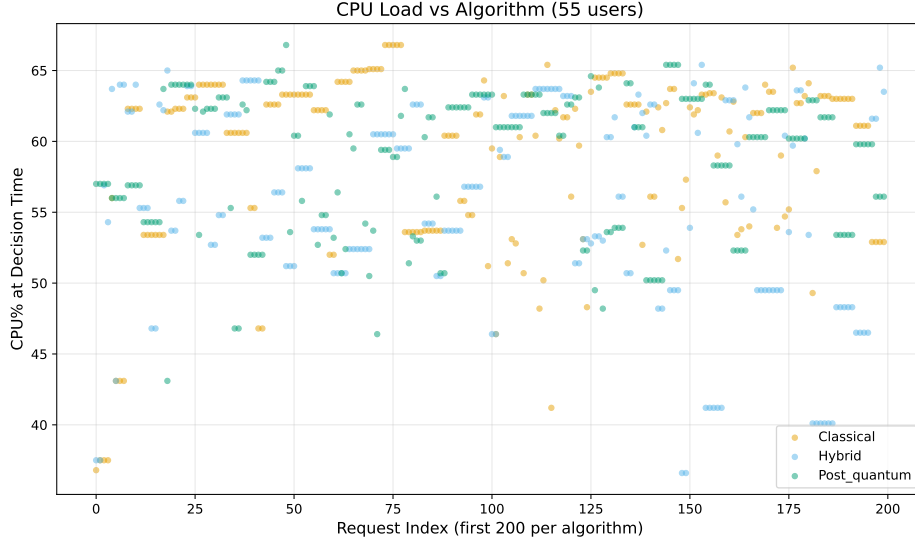


Fig. 5. CPU usage during decision-making as a function of the request index for 55 simultaneous users, which evaluates the first 200 decisions made by each algorithm. In all the cryptographic algorithms analyzed, the CPU usage was within the same range, revealing the multidimensional nature of the policy’s functioning.

Figure 6 shows the breakdown of the stacked approach for the last 1,000 requests for each of the four loads. The proportion of post-quantum requests is always the highest among the three types of choices, irrespective of the load, whereas the classical and hybrid portions are relatively low. This demonstrates that the high rates of post-quantum decisions shown in Figure 2 persist for all levels of sensitivity, rather than being limited to one level only, thus proving that the per-sensitivity Q-table is effective and consistent in its operation.

Figure 7 illustrates the curve of epsilon decay in the case of the medium-sensitivity Q-table with 55 users. Starting with a preset value of 0.30, the decay process exhibited a consistent decline toward a lower bound of 0.05. However, it is not completely linear; some spikes are visible on the graph and reflect re-exploration attempts launched upon encountering a trailing reward below the threshold. Despite this, it can be seen that the mechanism works properly; by the time approximately 200 requests pass, the decay comes to a halt.

The mean end-to-end latency of the HTTP and TLS (HTTPS) transports at each of the four specified load levels is illustrated in Figure 8. While there is only a slight overhead due to TLS in the low concurrency case (15 users) compared to HTTP, latency sharply increases as the load increases due to CPU-bound DistilBERT inference, and the overhead due to TLS decreases in a corresponding manner. With 55 and 80 users, the degradation characteristics of both transport types begin to coincide, indicating that the dominant bottleneck is the AI inference processing capacity, not the transport or encryption layer. However, despite this being true at all load levels, where there is still some TLS overhead, it still justifies the choice of adaptive policy because, with low-risk requests that are already being processed over an encrypted transport layer, the overhead can be reduced by using classical cryptography techniques.

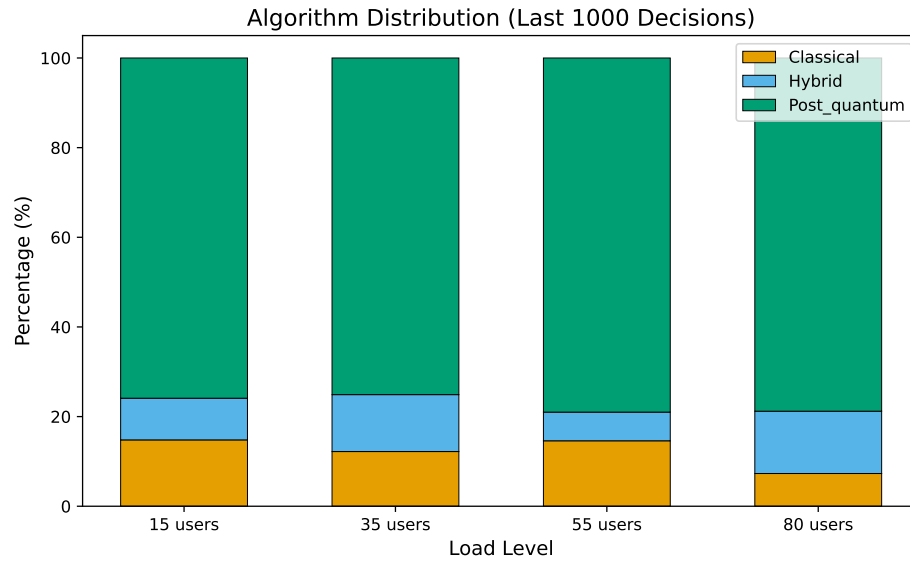


Fig. 6. Stacked distribution of the algorithms for the last 1,000 decisions at each concurrency level. Post-quantum algorithms always occupied the highest proportion.

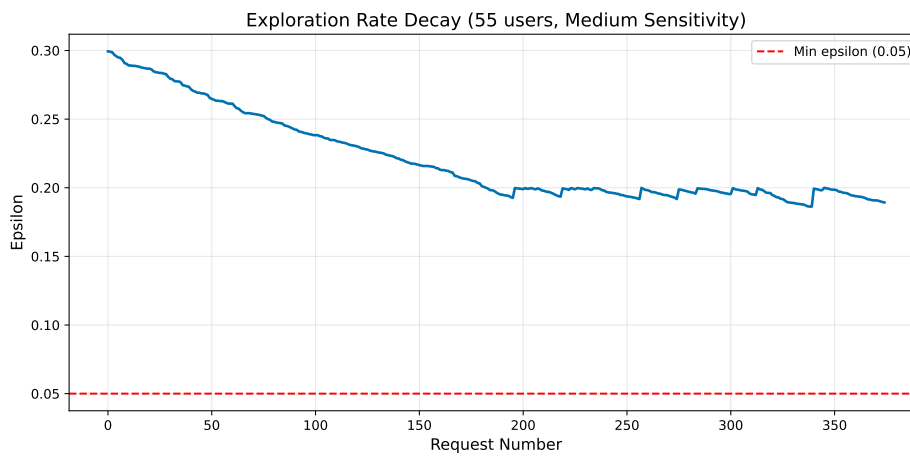


Fig. 7. Epsilon decay curve for medium-sensitivity Q-table with 55 simultaneous users. The short upward spikes represent adaptive re-exploration.

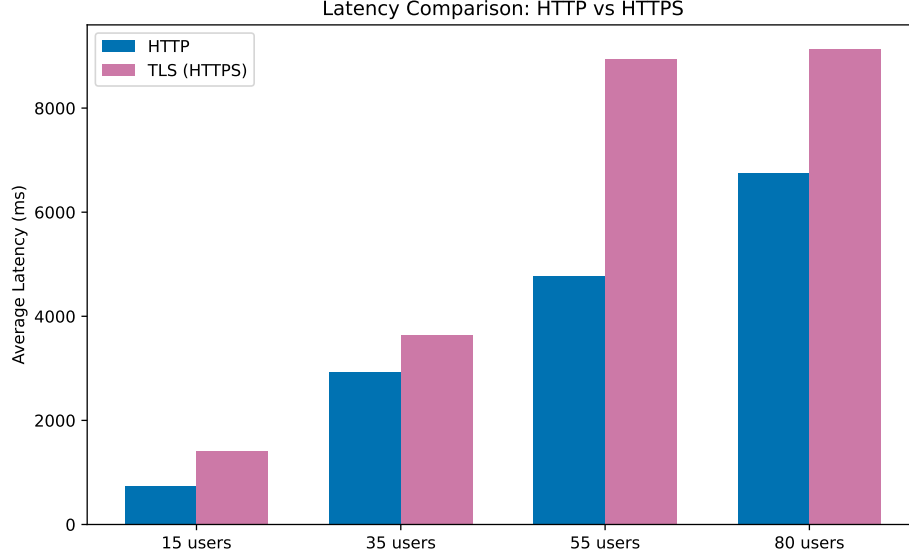


Fig. 8. Comparative study between the end-to-end latency for HTTP and TLS-based connections (HTTPS). The additional latency introduced by TLS is visible at all concurrency levels but is reduced in relative importance with increasing load.

4.3 Validation and System Performance Evaluation

Policy Correctness. Previous iterations of the model achieved post-quantum security for high-sensitivity traffic approximately 54.5% of the time. The per-sensitivity double Q-table architecture introduced in [23] solves the cross-sensitivity discrepancies encountered by previous systems. Figures 2 and 6 indicate that the current iteration of the model achieves a post-quantum selection rate higher than 75% for all loads; this security level is preserved even under CPU saturation due to a penalty of -0.20 applied anytime a non-post-quantum method is chosen for sensitive data.

Learning Convergence. Figure 3 illustrates convergence to a relatively stable reward score close to -0.14 after approximately 300 requests, indicating a solid exploitation strategy. Another method supporting this statement is experience replay [12], which minimizes the impact of consecutive updates of the Q-values.

Throughput and performance bottlenecks. Multidimensional decisions made by the policy rather than a straightforward threshold-based one are demonstrated in Figures 4 and 5; however, Figure 6 illustrates certain performance bottlenecks at 55 or more users that may be associated with queuing for DistilBERT model predictions (see Table 5) but not the cryptography layer, whose latency never exceeded 2.2 ms.

4.4 Discussions on Contributions

Per-Sensitivity Double Q-Tables. These four are the salient contributions of this study. First, the introduction of per-sensitivity Double Q-tables solves the issue of cross-sensitivity Q-value contamination by allowing different sensitivity levels to compute their best policy separately.

Dynamic Latency-Aware Security Bonus (DLASB). Second, the adoption of a dynamic security bonus, which scales based on the current latency $\beta \cdot (1 - L/\text{LATENCY_CAP})$, prevents the agent from foregoing its post-quantum security under stress without making an implausible commitment.

Weak Cryptography Penalty. Third, a weak cryptography penalty on high-sensitivity requests (-0.20) makes it impossible for the agent to satisfy this rule via latency considerations.

End-to-End Encrypted Channel. Lastly, the end-to-end encrypted connection achieved through ML-KEM-768 [14] guarantees that plaintext data is never sent in clear text, providing quantum resistance outside of the

transport layer. In particular, the comparison between the HTTP and TLS settings (Figure 8) shows the value of the adaptive strategy as it allows the choice of classical cryptography over post-quantum in case of low sensitivity and vice versa in case of high sensitivity. This allows the agent to defend against adversaries who intercept encrypted TLS messages and decrypt them with a future quantum computer, a scenario explicitly covered by the NIST PQC standards process [14, 15].

Limitations and Future Work. The primary limitation of this system lies in its CPU-bound inference process using DistilBERT, which can be addressed by implementing a GPU-based system or adopting a more efficient model. Furthermore, the end-to-end encryption channel employed in this system uses the server’s public key to encrypt the response key only as a demonstration; a production-grade system would require proper registration of a client key to achieve true decapsulation on the client side. Lastly, conducting a formal security analysis on the reward function – ensuring that penalties and bonuses generate the desired incentives for all state-action pairs – is another direction for further research.

5 Conclusion and Future Scope

This research has successfully designed, implemented, and evaluated an AI-driven adaptive cryptographic framework tailored for the security requirements of modern web services in the impending quantum era. As established in the literature, the emergence of fault-tolerant quantum computing and Shor’s algorithm presents an existential threat to classical RSA and ECC infrastructures. While NIST-standardized Post-Quantum Cryptography (PQC) such as ML-KEM (Kyber) and ML-DSA (Dilithium) provides robust resilience, its practical deployment is often hindered by significant increases in key sizes and handshake latency. This study addressed these challenges by moving beyond static security policies—which often lead to unnecessary performance degradation or inadequate protection—toward a model of intelligent cryptographic agility.

The core contribution of this work, the PQC-AI service, demonstrated that an online Reinforcement Learning agent (formulated as a contextual bandit with Double Q-learning) can effectively optimize the security-performance tradeoff. By maintaining per-sensitivity Double Q-tables, the system successfully mitigated cross-sensitivity contamination and maintained a post-quantum selection rate of over 75% across all load levels, even under CPU saturation. The experimental results proved that while PQC can increase handshake sizes by up to 7×, the total cryptographic overhead in an optimized implementation remained below 2.2 ms, which is negligible compared to the resource-heavy AI inference task. Furthermore, the microbenchmarks revealed that ML-KEM-768 offers significantly more efficient decryption (0.298 ms) than classical RSA-2048 (1.836 ms), establishing PQC as a viable high-performance alternative.

The hybrid implementation approach utilized in this project ensured backward compatibility and “defense-in-depth,” providing a shared secret that remains secure as long as one of the component algorithms (classical or PQC) remains unbroken. By integrating this hybrid model with AI-driven selection, the framework provided quantum-resistant data protection outside the transport layer, effectively mitigating “Harvest Now, Decrypt Later” risks for sensitive AI inference results.

Future Scope

While the current framework establishes a strong foundation, several avenues for future research remain:

- **Infrastructure Optimization:** The primary bottleneck identified was the CPU-bound nature of the DistilBERT model. Future iterations should transition to GPU-based inference or investigate ultra-lightweight architectures like TinyBERT to improve overall system throughput [8, 22].
- **Algorithm Expansion:** The repository should be updated to include the newly finalized HQC standard and other code-based alternatives like Classic McEliece for high-assurance use cases, enhancing the system’s cryptanalytic diversity.
- **Cross-Platform Portability:** Extending the framework to resource-constrained mobile and IoT devices is critical, as these platforms are highly sensitive to the energy and memory overheads of PQC [1].

- **Enhanced Entropy Sources:** Integrating Quantum Random Number Generators (QRNGs) into the key derivation process could further strengthen session security against randomness-related attacks [7].
- **Advanced Learning Architectures:** Investigating Model-Agnostic Meta-Learning (MAML) could reduce the time required for the RL agent to adapt to new network conditions or device profiles, further increasing system robustness [2].

In conclusion, this research confirms that AI-adaptive hybrid cryptography is not only feasible but essential for securing the next generation of intelligent digital infrastructure, ensuring that high-velocity web services remain both performant and resilient in a post-quantum world.

Bibliography

- [1] Abbasi, M., Cardoso, F., Váz, P., Silva, J., Martins, P.: A Practical Performance Benchmark of Post-Quantum Cryptography Across Heterogeneous Computing Environments. *Cryptography* 9, 32 (2025)
- [2] Ali, A.K.S., Raza, A., Arif, H., Hussain, A.A.: AI-Driven Dynamic Selection of Post-Quantum Algorithms for Mobile Application Security. *The Asian Bulletin of Big Data Management* 5(2), 51–62 (2025)
- [3] Alnahawi, N., Müller, J., Oupický, J., Wiesmaier, A.: A Comprehensive Survey on Post-Quantum TLS. *IACR CiC Cc1-2–5* (2024)
- [4] Benitez, C.: Mapping Quantum Threats: An Engineering Inventory of Cryptographic Dependencies. *arXiv preprint arXiv:2509.24623* (2025)
- [5] Cherkaoui Dekkaki, K., Tasic, I., Cano, M.-D.: Exploring Post-Quantum Cryptography: Review and Directions for the Transition Process. *Technologies* 12, 241 (2024)
- [6] Demir, E.D., Bilgin, B., Onbasli, M.C.: Performance Analysis and Industry Deployment of Post-Quantum Cryptography Algorithms. *arXiv preprint arXiv:2503.12952* (2025)
- [7] Balakrishnan, S., Uppalapati, N.R.K., Gokul Kumari, G., Khoshaim, L.S., Ramya, S., Gottimukkala, A.R.: AI-Adaptive Post-Quantum Cryptography for Secure and Fraud-Resilient Digital Payments. *Journal of Theoretical and Applied Information Technology* 104(2), 93–103 (2026)
- [8] Kumar, B.P., Ahmed, M.S., Sadanandam, M.: DistilBERT: A Novel Approach to Detect Text Generated by Large Language Models (LLM). *ResearchSquare* (2024)
- [9] Li, P., Chen, T., Liu, J.: Enhancing Quantum Security over Federated Learning via Post-Quantum Cryptography. In: 2024 IEEE 6th International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA), pp. 499–505. IEEE, Washington, DC (2024)
- [10] Liu, T., Ramachandran, G., Jurdak, R.: Post-Quantum Cryptography for Internet of Things: A Survey on Performance and Optimization. *arXiv preprint arXiv:2401.17538* (2024)
- [11] Mamatha, G.S., Dimri, N., Sinha, R.: Post-Quantum Cryptography: Securing Digital Communication in the Quantum Era. *arXiv preprint arXiv:2403.11741* (2024)
- [12] Mnih, V., et al.: Human-level control through deep reinforcement learning. *Nature* 518, 529–533 (2015)
- [13] National Institute of Standards and Technology (US): Module-lattice-based key-encapsulation mechanism standard. *NIST FIPS 203* (2024)
- [14] National Institute of Standards and Technology: Module-Lattice-Based Key-Encapsulation Mechanism Standard. *FIPS 203* (2024)
- [15] National Institute of Standards and Technology: Module-Lattice-Based Digital Signature Standard. *FIPS 204* (2024)
- [16] National Institute of Standards and Technology: Recommendation for Key-Derivation Methods in Key-Establishment Schemes. *SP 800-56C Rev. 2* (2020)
- [17] Premakumari, S.B.N., Sundaram, G., Rivera, M., Wheeler, P., Guzmán, R.E.P.: Reinforcement Q-Learning-Based Adaptive Encryption Model for Cyberthreat Mitigation in Wireless Sensor Networks. *Sensors* 25, 2056 (2025)
- [18] Sanh, V., Debut, L., Chaumond, J., Wolf, T.: DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019)
- [19] Souvatzidaki, K., Limnietis, K.: Post-Quantum Key Exchange in TLS 1.3: Further Analysis on Performance of New Cryptographic Standards. *Cryptography* 9, 73 (2025)
- [20] Stebila, D., Mosca, M.: Post-quantum key exchange for the internet and the open quantum safe project. In: *Proc. SAC*, pp. 14–37 (2016)
- [21] Sutton, R.S., Barto, A.G.: *Reinforcement Learning: An Introduction*, 2nd edn. MIT Press, Cambridge (2018)
- [22] Thomas, G.J.: Enhancing TinyBERT for Financial Sentiment Analysis Using GPT-Augmented FinBERT Distillation. *arXiv preprint arXiv:2409.18999* (2024)

- [23] van Hasselt, H., Guez, A., Silver, D.: Deep reinforcement learning with double Q-learning. In: Proc. AAAI, pp. 2094–2100 (2016)
- [24] Zafar, A., Iqbal, S.S.: Integrating code-based post-quantum cryptography into SSL TLS protocols through an interoperable hybrid framework. *Discov Computing* 28, 202 (2025)
- [25] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proc. NAACL-HLT, pp. 4171–4186 (2019)
- [26] Socher, R., et al.: Recursive deep models for semantic compositionality over a sentiment treebank. In: Proc. EMNLP, pp. 1631–1642 (2013)